

QChain Audit-Readiness Report

A retrospective on a self-imposed engineering discipline applied to a post-quantum research
blockchain

QChain project

2026-05

Contents

1	Abstract	3
2	What this project is, and what it isn't	4
3	The audit-readiness arc	4
3.1	Audit-followup hardening (5 passes)	4
3.2	Mixer timing-attack defense (M-timing)	5
3.3	The first audit package	5
3.4	Differential AIR validation (two phases)	5
3.5	Property-based testing with Hypothesis	6
3.6	Documentation cleanup and the threat model	6
3.7	Wallet key encryption at rest (T21)	6
3.8	Rate limiting and DoS hardening (T15, T22, T23)	7
3.9	Wallet note lifecycle (ROADMAP 1.6)	7
3.10	Dashboard authentication (T22 full closure)	7
4	The current state, in numbers	8
5	Self-disclosed findings	9
5.1	Coinbase inflation (audit-followup pass)	9
5.2	Transparent transaction replay (property-testing pass)	9
5.3	Same-height-fork ping-pong (rate-limiting pass)	10
5.4	Stale wallet docstring (note-lifecycle pass)	10
5.5	Stale THREAT-MODEL.md honest-gaps summary (auth pass)	10
6	Honest standing limitations	10
7	For external auditors, researchers, or collaborators	11
7.1	If you have one hour	11
7.2	If you have one day	12
7.3	If you want to actually audit the AIR	12
7.4	If you want to find more self-disclosure-quality bugs	12

8	What the project would do differently	12
9	Closing	13

1 Abstract

QChain is a research blockchain that combines post-quantum signatures (CRYSTALS-Dilithium), quantum-randomness-driven leader election (IBM Quantum Network QRNG), and a hand-rolled zk-STARK proof system (Goldilocks AIR via Winterfell) for shielded payments. Beyond its cryptographic primitives, the project is unusual for the engineering discipline applied to it: every component carries explicit honest- scope notes, every defended attack is paired with code and tests, and every known limitation is documented as a numbered threat with status markers. This report is a retrospective on the audit-readiness arc that produced this state — what work was done, what was found, and what would help an external auditor or research collaborator engage next. The codebase at the time of writing contains 443 tests across three implementation layers, 36 maintained Markdown documents, and a threat model with 23 numbered entries. Five bugs and two documentation defects were found and fixed by the project itself during this arc; this report includes all of them. The intended audience is external security researchers, audit firms, and grant reviewers who may be considering deeper engagement with the project.

2 What this project is, and what it isn't

QChain is a **research demo**. It runs a working chain end-to-end: TCP peer-to-peer gossip, mempool, fork resolution, block production, multiple shielded-payment pools, and a live FastAPI dashboard. It has been built incrementally over roughly two dozen milestones, with each milestone leaving behind a Markdown design document, an implementation, and a test suite. It is intended for research use — exploring how quantum-resistant primitives and STARK-based shielding interact in a single coherent system — not for production deployment.

What it is not:

- It is not a peer-reviewed cryptographic construction. The primitives are well-known (Dilithium, Rescue-Prime, Winterfell-based AIRs), but the *combination* and the AIR-level circuits for shielded payments have not been formally analyzed by external cryptographers. This report is part of preparing for that analysis, not a substitute for it.
- It is not a production blockchain. There is no BFT for malicious- quorum proof-of-stake, no peer authentication, no TLS by default, no key-management hygiene beyond optional wallet encryption-at-rest, and no governance model. The dashboard binds to 127.0.0.1 and the protocol assumes peers are at worst lazy, not adversarial in the ways a real public chain must withstand.
- It is not a paper-ready research artifact. The AIRs, the STARK parameter choices, and the multi-pool composition story have not been written up for a venue with peer review. They are designed to be **possible to write up** — every claim has code and tests behind it — but a researcher who wants to publish on the system would need to do their own evaluation and framing.

What the project has done, and what this report describes, is **make itself legible to someone who would want to engage with it**: an external auditor reviewing the protocol design, a grant reviewer evaluating the work for funding, or a researcher considering collaboration. The audit-readiness arc was a deliberate decision to spend several passes of engineering time on the boring infrastructure of “describe what’s defended, find what was hiding, write down the limitations” rather than on adding more features.

3 The audit-readiness arc

Over a sequence of passes between roughly late 2025 and mid 2026, the project executed an audit-readiness program with the following structure. Each pass produced an implementation, a test suite, and a Markdown design document; passes are described chronologically below.

3.1 Audit-followup hardening (5 passes)

The first phase identified gaps in the implementation that an external auditor would likely find immediately. These passes added:

- **Replay-side validation for block reward inflation.** The original `is_valid()` did not check the coinbase amount against the expected block reward plus in-block fees. A malicious miner could inflate their reward without detection. The fix computes the expected reward (`BLOCK_REWARD + sum(anon_fees) + sum(stark_fees)`) and verifies the block’s coinbase matches. Closes T4 in the threat model.

- **PERSISTENCE_VERSION field with explicit migration semantics.** Wallet and chain files now carry a version number; the loader refuses to load a file with an unknown version. This avoids the silent-corruption failure mode of schema drift.
- **PoW validation in `is_valid()`.** The replay path now checks difficulty for proof-of-work-mined blocks, not just the production path.
- **Coverage tests for the above.** Fourteen new tests covering the fixes, plus regression-style tests that demonstrate the bug’s original behavior.

A document, `AUDIT-NOTES.md`, accompanies these passes describing each finding and its remediation in detail.

3.2 Mixer timing-attack defense (M-timing)

A separate pass addressed T13 (mixer same-block linkability). The attack: if a withdrawal could be made against the current pool root in the same block as a deposit, the deposit-to-withdrawal pairing might be inferable from timing. The defense: withdrawals must anchor against a *historical* mixer root at least `MIXER_WITHDRAWAL_DELAY` blocks behind the chain tip. This implementation required:

- Tracking historical roots in `Blockchain.mixer_root_history`
- Modifying the STARK proof construction to bind the anchored root
- Validation logic that recognizes when a proof’s anchor is too recent

Closes T13. T14 (statistical timing analysis across many blocks) remains [NOT DEFENDED] and is documented as such.

3.3 The first audit package

A document, `AUDIT-PACKAGE.md`, was assembled at this point as a single entry-point for an external auditor. It enumerates ten major security claims (C1–C10), each cross-referenced to the relevant code and test files. This is a deliberately auditor-shaped artifact: it does not try to teach the reader the codebase; it tries to make each defended claim *findable* and *checkable*.

3.4 Differential AIR validation (two phases)

The zk-STARK circuit is the part of the system that an auditor would trust the least without independent corroboration. AIRs are easy to write subtly wrong: an off-by-one in a transition constraint, a mis-ordered round in a hash, or an incomplete boundary constraint can produce a circuit that accepts proofs of false statements while appearing to work on positive examples.

The differential AIR program runs the trace produced by the AIR through an *independent native computation* — Python code that re-derives the same intermediate values from first principles — and asserts that the trace’s contents match the native computation row by row.

- **Phase 1** (12 tests) cross-checks boundary content (root, nullifier, leaf, `output_leaf`) against an independent native computation.
- **Phase 2** (9 tests) adds a full Python re-execution of the Rescue-Prime round function over every interior row of every active block of the trace, catching bugs where boundary outputs happen to match by coincidence but interior state is wrong. The constants for Rescue-Prime

were extracted directly from `winter-crypto-0.8.3` so the differential test does not share any implementation with the trace builder it is checking.

This is the most aggressive technical defense in the project — it specifically targets the class of “the circuit is wrong in a way that all positive examples still work” bug. No such bug has been found, but the *capacity to find one* if it existed is what an auditor would need to trust the AIR.

3.5 Property-based testing with Hypothesis

A Hypothesis-driven test pass executed randomized sequences of valid chain operations and asserted invariants after each. The interesting finding: a transparent-transaction replay bug. Specifically, the mempool accepted re-submission of a transaction that had already been mined into a block. Property test P6a failed; the fix added `Blockchain.mined_txids: Set[str]` checked at admission.

This is the second self-found bug in the project’s history (after coinbase inflation). It is documented in `PROPERTY-TESTING-README.md`.

3.6 Documentation cleanup and the threat model

A focused pass rewrote the project README, added a `DOCS.md` reading-order guide, and consolidated the threat model into a single `THREAT-MODEL.md` document with explicit status markers for each numbered threat. The status markers are:

- `[FORMAL]` — defended under a standard cryptographic assumption (e.g. Dilithium EUF-CMA, hash collision resistance). Nine threats.
- `[FORMAL, MODULO]` — defended under a standard assumption *and* an additional project-specific assumption (e.g. AIR correctness, pool composition). Four threats.
- `[DEFENDED]` — defended by application-layer mechanism with code and tests. Five threats.
- `[HEURISTIC]` — defended by partial or best-effort mechanism that is not guaranteed to catch every case. Two threats.
- `[HEURISTIC, INCOMPLETE]` — partially defended; specific gap documented. One threat.
- `[NOT DEFENDED]` — explicitly out of scope; documented limitation. Two threats.

This taxonomy is unusual. Most threat models in the academic literature use binary “defended / not defended” labels. The richer taxonomy is intended to be **honest** about the actual epistemic status of each defense — a `[FORMAL]` claim and a `[HEURISTIC]` claim should not be presented to a reader as equally strong.

3.7 Wallet key encryption at rest (T21)

The wallet’s Dilithium secret key was stored base64-encoded in plaintext JSON. This pass added optional encryption-at-rest using argon2id (OWASP 2023 baseline parameters: 64 MiB memory, 3 iterations, 4 parallelism) for key derivation and AES-256-GCM for authenticated encryption. Sixteen tests cover roundtrip, wrong-passphrase rejection, tampering detection, backward compatibility with legacy plaintext files, and forward-compatibility-rejection of unknown wallet formats.

The interface is `wallet.save(path, passphrase=...)` and `wallet.load(path, passphrase=...)`. Passphrase is optional; omitted-passphrase saves write the legacy format. This was a deliberate design choice — making encryption mandatory would have broken every existing test in a way that conflated “the wallet supports encryption” with “the wallet refuses to operate without it.” The latter is a policy decision, not a primitive.

Closes T21.

3.8 Rate limiting and DoS hardening (T15, T22, T23)

A single pass added three independent mechanisms:

1. **Per-peer per-message-type rate limits on the network layer.** A small `SlidingWindowRateLimiter` primitive (~80 lines) gates `Node._handle_message` with separate buckets for tx-class messages (100/sec/peer), block messages (50/sec/peer), sync messages (50/sec/peer), and hello messages (10/sec/peer). Closes T15.
2. **Per-IP per-method rate limits on the dashboard.** FastAPI middleware applies separate buckets for POST (5/sec/IP) and GET (50/sec/IP) endpoints. Partially closes T22.
3. **MAX_BLOCK_TX_COUNT = 10_000 cap.** Checked at both the admission path (`Node._handle_new_block`) and the replay path (`Blockchain.is_valid()`), matching the project's existing admission-vs-replay-consistency pattern. Closes T23.

The pass produced a side discovery: the rate limiter exposed a pre-existing protocol bug. When two nodes had competing blocks at the same height, the original `_handle_blocks` re-requested the full chain even on same-height alternatives, creating an infinite `get_blocks/blocks` ping-pong. The rate limiter throttled this loop, which caused a previously-flaky test (`test_concurrent_blocks_resolved_by_extension`) to fail deterministically. A one-line guard in `_handle_blocks` — “only re-request if the alternative chain is longer than ours” — fixed the loop. The test is now reliably fast (~1 second) instead of intermittently failing at a 6-second timeout.

This is the third self-found bug.

3.9 Wallet note lifecycle (ROADMAP 1.6)

The ROADMAP item was to “add persistence for wallet shielded notes,” quoting the wallet module docstring that said notes were “stored in-memory only.” On disk inspection, this was found to be untrue: notes had been persisted since the earlier persistence pass, and a test (`test_persistence_wallet_shielded_notes_roundtrip`) had been covering that roundtrip the whole time. The ROADMAP text had quoted a stale docstring as the problem statement.

The pass pivoted to the actual gap, which was different: the wallet had no way to determine whether the notes it thought it owned were *actually* on-chain. A note added by `create_mixer_deposit()` enters the wallet immediately, before the deposit is gossiped, validated, or mined. If the deposit never lands, the wallet carries dead-weight forever.

The pass added three helpers: `reconcile_with_chain()` (read-only classifier returning a structured report of confirmed vs. pending notes), `reconcile_summary()` (one-line human-readable view), and `prune_pending_notes()` (opt-in destructive cleanup). The stale docstring was fixed.

This is the fourth self-found defect — a documentation defect rather than a behavior bug.

3.10 Dashboard authentication (T22 full closure)

Rate limiting alone had only partially closed T22. The final pass in this arc added bearer-token authentication to all `/api/*` and `/ws` endpoints. Token comparison uses `hmac.compare_digest` for constant-time semantics. Token sources, in order of precedence: CLI flag, environment variable, or

auto-generated at startup and printed to stdout. Auth middleware runs *before* the rate limiter, so unauthenticated requests get a clean 401 without consuming legitimate users’ rate-limit budget — this is exercised by a specific test.

A fifth self-found defect was discovered during this pass: the `THREAT-MODEL.md` document’s bottom-of-file “honest gaps” summary still listed several threats as `[NOT DEFENDED]` even though prior passes had defended them. The summary was updated to reflect current state.

Closes T22.

4 The current state, in numbers

Metric	Value
Total tests (all layers)	443
qstark (Rust AIR core) tests	110
qstark_py (PyO3 bindings) tests	21
QChain Python tests	312
Markdown documents in the repo	36
Numbered threats in <code>THREAT-MODEL.md</code>	23
Threats <code>[DEFENDED]</code> or stronger	20
Threats <code>[NOT DEFENDED]</code>	2
Self-found bugs/defects in this arc	5

The full threat-status table:

ID	Threat	Status
T1	Spending without authorization (transparent)	<code>[FORMAL]</code>
T2	Double-spending (transparent)	<code>[FORMAL]</code>
T3	Coin minting (transparent)	<code>[FORMAL]</code>
T4	Block reward inflation by malicious miner	<code>[FORMAL]</code>
T5	Forged block (any node)	<code>[FORMAL]</code>
T6	STARK pool unauthorized spend	<code>[FORMAL, MODULO]</code>
T7	STARK pool double-spend	<code>[FORMAL]</code>
T8	STARK pool inflation/destruction	<code>[FORMAL, MODULO]</code>
T9	STARK pool stale-root attack	<code>[FORMAL, MODULO]</code>
T10	Mixer pool unauthorized withdraw	<code>[FORMAL, MODULO]</code>
T11	Mixer pool inflation across denomination boundary	<code>[FORMAL]</code>
T12	Mixer denomination-set partition	<code>[HEURISTIC]</code>
T13	Mixer same-block linkability	<code>[DEFENDED]</code>
T14	Mixer timing analysis across blocks	<code>[NOT DEFENDED]</code>
T15	Mixer DoS via gossip flood	<code>[DEFENDED]</code>
T16	M4 anon pool unauthorized spend	<code>[FORMAL]</code>
T17	M4 anon pool double-spend	<code>[FORMAL]</code>
T18	Persistence corruption	<code>[HEURISTIC]</code>
T19	Cross-block-format-version persistence	<code>[HEURISTIC, INCOMPLETE]</code>
T20	Replay across networks	<code>[NOT DEFENDED]</code>

ID	Threat	Status
T21	Wallet key compromise	[DEFENDED]
T22	Dashboard endpoint abuse	[DEFENDED]
T23	Memory exhaustion via large block	[DEFENDED]

The “[FORMAL, MODULO]” qualifier indicates that the defense holds under a standard cryptographic assumption *and* an additional project-specific assumption — most often, AIR correctness. An external auditor’s job, with respect to those threats, is to validate the project-specific assumption. The differential AIR work (the differential-AIR section) is the project’s best attempt to build evidence for AIR correctness without external review; it is not a substitute for external review.

5 Self-disclosed findings

A useful signal about a project is what it has found and disclosed about itself. During the audit-readiness arc, the project found and fixed five issues that no external party identified for it. They are all documented in the repo; this section consolidates them.

5.1 Coinbase inflation (audit-followup pass)

Severity (counterfactual): Critical in production. A malicious miner could include a coinbase transaction with any amount they chose, since the replay path (`Blockchain.is_valid()`) did not validate the coinbase value against the expected `BLOCK_REWARD + fees`.

Discovery: Found by deliberate audit-readiness review of every `is_valid()` codepath, comparing it to the corresponding `mine_pending()` enforcement.

Fix: Added expected-reward computation in `is_valid()` plus direct equality check on the block’s coinbase amount.

Test: A regression test that constructs a block with an inflated coinbase and asserts `is_valid()` returns False.

Documented in: AUDIT-NOTES.md.

5.2 Transparent transaction replay (property-testing pass)

Severity (counterfactual): High. A confirmed transparent transaction could be re-submitted to the mempool and mined again, producing a duplicate debit/credit. The transaction’s nonce was not sufficient to prevent this because the mempool didn’t track which nonces had been mined.

Discovery: Found by Hypothesis-driven random valid-operation sequences. The property test asserted “after a transparent tx is mined, re-submitting it must be rejected.” The assertion failed.

Fix: Added `Blockchain.mined_txids: Set[str]` (with a matching load-time reconstruction from `chain.blocks`), checked at admission.

Test: Property P6a in `test_chain_properties.py` plus deterministic unit tests in `test_chain.py`.

Documented in: PROPERTY-TESTING-README.md.

5.3 Same-height-fork ping-pong (rate-limiting pass)

Severity (counterfactual): Medium. Network performance bug rather than a security bug — but if a real network ever experienced sustained same-height forks (a legitimate scenario during contested mining), peers would saturate each other with sync traffic.

Discovery: The rate limiter throttled what turned out to be an infinite loop; the `test_concurrent_blocks_resolved` test, which was known-flaky with a 6-second timeout, became deterministically failing once the limiter was in place. Investigation identified the underlying loop.

Fix: A one-line guard in `_handle_blocks` — re-request from peers only if the received alternative chain is *longer* than the local chain. Same-length forks no longer trigger re-requests.

Test: `test_concurrent_blocks_resolved_by_extension` is now reliably fast (~1 second) instead of flaky. The fix also matters in production: it prevents quadratic gossip cost during fork resolution.

Documented in: `RATE-LIMITING-README.md`.

5.4 Stale wallet docstring (note-lifecycle pass)

Severity: Documentation defect; no behavioral bug. The wallet module’s top-of-file docstring said shielded notes were “stored in-memory only and not persisted by `save()`” — which had been untrue for several milestones, since the persistence pass had added note persistence.

Discovery: The ROADMAP 1.6 entry quoted this stale docstring as its problem statement. On opening the wallet module to implement the fix, the persistence was found to already exist.

Fix: The docstring was rewritten. The roadmap item was retargeted to the *actual* gap (no way to detect orphaned pending notes), which became the lifecycle helpers described in the note-lifecycle section.

Documented in: `WALLET-NOTE-LIFECYCLE-README.md`, which discusses the discovery explicitly.

5.5 Stale `THREAT-MODEL.md` honest-gaps summary (auth pass)

Severity: Documentation defect. The “what’s still open” summary at the bottom of `THREAT-MODEL.md` listed several threats as `[NOT DEFENDED]` that had been defended by intermediate passes.

Discovery: Found when updating T22’s status during the auth pass; the bottom-of-file summary was inconsistent with the per-threat sections above it.

Fix: Updated the summary to reflect actual current status of T13, T15, T21, T22, T23.

Documented in: `DASHBOARD-AUTH-README.md`.

6 Honest standing limitations

These are the gaps the project *knows about and has chosen not to close in this scope*. They are listed for completeness; an external auditor or collaborator should not have to discover them.

- **No formal proof of AIR correctness.** The differential AIR work (the differential-AIR section) tests for one specific class of bug — wrong interior computation that happens to

produce right boundary output — but does not constitute a proof. Several threats marked [FORMAL, MODULO] ride on AIR correctness as their non-formal assumption.

- **No BFT for malicious-quorum PoS (A4).** The PoS mode is for demo purposes; a single validator’s QRNG-derived randomness is the only source of leader election. A real PoS would need fault tolerance against a fraction of malicious validators.
- **No on-load chain validation (T18).** `Blockchain.load()` does not call `is_valid()` on the loaded chain. A corrupt-but-parseable persistence file would be accepted without the protocol-level invariants being checked. Closing this is a single-session item.
- **No version-tagged persistence beyond the wallet (T19).** The `PERSISTENCE_VERSION` mechanism exists for wallets but is incomplete for chain files. Schema drift in chain format would currently produce parse errors at best, silent misbehavior at worst.
- **No statistical timing-attack defense across many blocks (T14).** The same-block linkability defense (T13) only addresses a single block’s worth of correlation. An adversary with long-term observation could still infer deposit-to-withdrawal pairings via statistical analysis. A real anonymity-preserving system would need additional countermeasures (mixing pools, delayed processing, network-layer privacy) that are out of scope.
- **No TLS at any layer.** The dashboard’s bearer token and the network protocol’s messages all travel in cleartext. Production deployments should put TLS-terminating reverse proxies in front of both.
- **No replay protection across networks (T20).** Transactions on one chain instance can be replayed onto another. This isn’t relevant for single-network use cases (research demo, single test environment) but would matter for a multi-network deployment.
- **Wallet note recovery requires the wallet file.** The chain holds only the leaf commitments of shielded notes; the spending witnesses live only in the wallet. If a wallet file is lost, the notes are not recoverable from the chain alone. An off-chain backup mechanism is the standard mitigation; out of scope here.
- **Single human reviewer at each pass.** No external eyes have reviewed the code or the design before this writeup. Self-review has limits, particularly for subtle protocol-level bugs. This is the gap the project is now trying to close by engaging external collaborators.

7 For external auditors, researchers, or collaborators

If you are reading this report because you may engage with the project, the following pointers are intended to lower your time-to- useful-feedback.

7.1 If you have one hour

Read `AUDIT-PACKAGE.md`. It is the single-entry-point document for auditors, listing the ten main security claims (C1–C10) with cross-references to code and tests. After that, scan `THREAT-MODEL.md` for the per-threat status. Together those two documents cover roughly 80% of the project’s security-relevant surface.

7.2 If you have one day

After the above, read the five “phase” READMEs in the order they were written:

1. `AUDIT-NOTES.md` (initial audit-followup hardening)
2. `DIFFERENTIAL-AIR-PHASE2-README.md` (AIR validation)
3. `PROPERTY-TESTING-README.md` (Hypothesis pass + replay bug)
4. `WALLET-KEY-ENCRYPTION-README.md` (T21)
5. `RATE-LIMITING-README.md` (T15, T22, T23 + protocol fix)
6. `DASHBOARD-AUTH-README.md` (T22 full closure)

This is the audit-readiness arc in narrative form. Each document includes an honest-scope section. `DOCS.md` provides a complete reading-order guide if you want to go deeper.

7.3 If you want to actually audit the AIR

The single most valuable external contribution would be an independent review of the STARK circuits for shielded payments. The relevant artifacts:

- `qstark/` (Rust) — the AIR implementation using Winterfell
- `qstark/tests/` — 110 tests covering the AIR
- `qchain/crypto/anon_stark.py` — the corresponding Python-side proof construction
- `qchain/tests/test_differential_air_phase2.py` — the project’s best internal evidence that the trace contents match an independent native computation

The project has not been able to convince itself that the AIR is *correct* — only that several specific incorrect-AIR failure modes would have been caught. This is the highest-leverage place for a formal-methods specialist or cryptographer to engage.

7.4 If you want to find more self-disclosure-quality bugs

The five bugs in the self-disclosed findings section were found by mechanisms the project still runs: deliberate audit-style review of replay paths (4.1), property- based testing (4.2), regression-test friction (4.3), and document- versus-code inconsistency review (4.4, 4.5). Extending any of these would likely find more:

- The Hypothesis test corpus could be deepened to cover mixer and STARK-pool flows more thoroughly. Currently it focuses on transparent transactions and basic chain invariants.
- The differential AIR work could extend to other AIRs (mixer withdrawal, anon-pool spend) that currently have only Phase 1 coverage.
- The audit-style replay-path review could be re-run on the network-layer code, which received less of this kind of attention than the chain-layer code.

8 What the project would do differently

In the spirit of honest self-assessment, several things would have been better if done earlier.

- **The [FORMAL] / [DEFENDED] / [HEURISTIC] / [NOT DEFENDED] status taxonomy should have been introduced at milestone M1, not at the documentation-cleanup pass.** Many milestone documents from earlier in the project have language that doesn’t

disambiguate between these — “the chain rejects this” can mean any of them. Retrofitting the taxonomy was time-consuming.

- **Property-based testing should have been part of the test suite from the start.** The transparent-tx replay bug had been latent for a long time and would have been caught by Hypothesis on day one. The project’s habit of writing positive-example tests is insufficient for invariant-style bugs.
- **Differential testing of AIRs should have been built before the AIRs were considered “done.”** It was added retroactively. Doing it concurrently would have caught any incorrect-AIR bugs before downstream code depended on the AIR’s output shape.
- **The ROADMAP should have been kept in sync with the code.** The ROADMAP 1.6 item quoted a stale docstring as its problem statement because the ROADMAP author did not re-read the code. A “disk check before scoping” step was added to the working discipline after this discovery and has held since.
- **The threat model’s bottom-of-file summary should be machine-generated.** The fifth self-found defect (the threat-model summary item) was a document going out of sync. A script that re-generates the summary from the per-threat sections would have made the inconsistency unrepresentable. A small follow-up.

9 Closing

The audit-readiness arc described in this report is engineering, not research. The interesting cryptographic ideas in QChain are not themselves novel — Dilithium, Rescue-Prime, STARK-based shielding, quantum-derived randomness are all known constructions. What may be novel about the project is the discipline applied to documenting, testing, and self-disclosing its limitations.

A research artifact that says “this is what we built and here are all the ways it would fail” is different from a research artifact that says “this is what we built.” The former is less impressive in a presentation slide but more useful to an auditor, a collaborator, or another researcher considering whether to build on the work. The project has spent meaningful time on the former mode in preference to the latter, and this report is meant to consolidate that work into something an external reader can engage with.

The project is now ready for external engagement. It is not ready for production deployment — and it does not claim to be. The honest gaps in the honest-limitations section are exactly the items that an external audit, a follow-up cryptography paper, or a more serious deployment effort would need to close.

The repository is available at <https://github.com/sil714/qchain>. All documents referenced in this report are in the repository root or one directory deep; `DOCS.md` provides a reading-order guide.